

Package ‘mSigAct’

May 3, 2024

Title Mutational Signature Activity Analysis ('mSigAct')

Version 3.0.1

Author Steve Rozen, Alvin Wei Tian Ng, Arnoud Boot, Nanhai Jiang

Maintainer Steve Rozen <steverozen@gmail.com>

Description Analyze the “activities” of mutational signatures in one or more mutational spectra. 'mSigAct' stands for mutational Signature Activity. mSigAct uses a maximum likelihood approach to estimate (conservatively) whether there is evidence that a particular set of mutational signatures is present in a spectrum. It can also determine a *minimal* subset of signatures needed to plausibly reconstruct an observed spectrum.

License GPL-3

URL <https://github.com/steverozen/mSigAct>

BugReports <https://github.com/steverozen/mSigAct/issues>

Encoding UTF-8

Language en-US

Depends R (>= 4.0),

RoxygenNote 7.2.3

biocViews

Imports cosmicsig,
dplyr,
gtools,
ICAMS (>= 3.0.6),
nloptr,
mSigTools,
PCAWG7 (>= 0.1.3),
philentropy,
quadprog,
stats,
sets,
tibble,
utils

Remotes github::steverozen/ICAMS@v3.0.6-branch,
github::steverozen/PCAWG7@v0.1.3-branch

Suggests BSgenome.Hsapiens.1000genomes.hs37d5,
devtools,

htmlwidgets,
ipc,
knitr,
profvis,
rmarkdown,
testthat ($\geq 2.1.0$),
usethis

R topics documented:

cossim	2
DefaultManyOpts	3
ExposureProportions	4
MAPAssignActivity	5
PresenceAttributeSigActivity	7
ReconstructSpectrum	9
SignaturePresenceTest	10
SparseAssignActivity	11

Index	14
--------------	-----------

cossim	<i>Cosine similarity with useful argument types</i>
--------	---

Description

Cosine similarity with useful argument types

Usage

```
cossim(v1, v2)
```

Arguments

v1	A vector or single-column matrix
v2	A vector or single-column matrix

Examples

```
spectrum <- PCAWG7::spectra$PCAWG$SBS96[, 1, drop = FALSE]
SBS96.sigs <- cosmicSig::COSMIC_v3.2$signature$GRCh37$SBS96
exposure <- PCAWG7::exposure$PCAWG$SBS96[, 1, drop = FALSE]
reconstructed.spectrum <- ReconstructSpectrum(sigs = SBS96.sigs,
                                              exp = exposure,
                                              use.sig.names = TRUE)
cosine <- cossim(spectrum, reconstructed.spectrum)
```

DefaultManyOpts	<i>Set default options for many functions, especially nloptr</i>
-----------------	--

Description

Set default options for many functions, especially [nloptr](#)

Usage

```
DefaultManyOpts(likelihood.dist = "neg.binom", spectra = NULL)
```

Arguments

<code>likelihood.dist</code>	The probability distribution used to calculate the likelihood, can be either "multi-nom" (multinomial distribution) or "neg.binom" (negative binomial distribution).
<code>spectra</code>	The catalog (matrix) to analyze. This could be an ICAMS catalog or a numerical matrix.

Value

A list with the following elements

global.opts A sub-list with several options for [nloptr](#), q.v., for the global optimization phase.

local.opts A sub-list with several options for [nloptr](#), q.v., for the local optimization phase.

nbinom.size Only appearing if `likelihood.dist = "neg.binom"`. The dispersion parameter for the negative binomial distribution; smaller is more dispersed. See [NegBinomial](#).

trace If > 0 print progress messages.

global_eval_f The objective function for the global optimization phase.

local_eval_f The objective function for the local optimization phase.

local_eval_g_ineq The inequality constraint function for the local optimization phase.

likelihood.dist The probability distribution used to calculate the likelihood.

Examples

```
my.opts <- DefaultManyOpts()
my.opts$trace <- 10
```

MAPAssignActivity	<i>Find Maximum A Posteriori (MAP) assignment of signature exposures that explain multiple spectra</i>
-------------------	--

Description

This function also can do sparse assignment by specifying `use.sparse.assign = TRUE`.

Usage

```
MAPAssignActivity(
  spectra,
  sigs,
  output.dir,
  sigs.presence.prop = NULL,
  max.level = 5,
  p.thresh = 0.05,
  m.opts = DefaultManyOpts(),
  num.parallel.samples = 5,
  mc.cores.per.sample = min(20, 2^max.level),
  progress.monitor = NULL,
  seed = NULL,
  max.subsets = 1000,
  use.sparse.assign = FALSE,
  use.forward.search = FALSE,
  drop.low.mut.samples = TRUE,
  use.sig.presence.test = FALSE,
  save.files = TRUE
)
```

Arguments

<code>spectra</code>	The spectra (multiple spectra) to be reconstructed.
<code>sigs</code>	A numerical matrix, possibly an ICAMS catalog.
<code>output.dir</code>	Directory path to save the output file.
<code>sigs.presence.prop</code>	The proportions of samples that contain each signature. A numerical vector (values between 0 and 1), with names being a subset of <code>colnames(sigs)</code> . See ExposureProportions for more details.
<code>max.level</code>	The maximum number of signatures to try removing.
<code>p.thresh</code>	If the p value for a better reconstruction with a set of signatures (as opposed to without that set of signatures) is > than this argument, then we can use exposures without this set.
<code>m.opts</code>	See DefaultManyOpts .
<code>num.parallel.samples</code>	The (maximum) number of samples to run in parallel. On Microsoft Windows machines it is silently changed to 1. Each sample in turn can require multiple cores, as governed by <code>mc.cores.per.sample</code> .

<code>mc.cores.per.sample</code>	The maximum number of cores to use for each sample. On Microsoft Windows machines it is silently changed to 1.
<code>progress.monitor</code>	Function called at the start of each new level (number of signatures to try excluding). Must take named arguments <code>value</code> and <code>detail</code> , and no others. Designed for a AsyncProgress progress bar function.
<code>seed</code>	Random seed; set this to get reproducible results. (The numerical optimization is in two phases; the first, global phase might rarely find different optima depending on the random seed.)
<code>max.subsets</code>	This argument provides a way to heuristically limit the amount of time spent by this function. Larger values of this argument will tend to allow longer running times. The algorithm successively tries to remove all subsets of 1 signature, 2 signatures, 3 signatures, etc., down to <code>max.level</code> . (Not every subset is tested at each level; if a subset was already found to be necessary the algorithm does not test supersets of that subset.) If at any level the algorithm needs to test more than <code>max.subsets</code> this function will not proceed.
<code>use.sparse.assign</code>	Whether to use sparse assignment. If <code>TRUE</code> , arguments designed for Maximum A Posteriori assignment such as <code>sigs.presence.prop</code> will be ignored.
<code>use.forward.search</code>	Whether to use forward search to find the minimal number of signatures to optimally reconstruct the spectrum.
<code>drop.low.mut.samples</code>	Whether to exclude low mutation samples from the analysis. If <code>TRUE</code> , samples with SBS total mutations less than 100, DBS or ID total mutations less than 25 will be dropped.
<code>use.sig.presence.test</code>	Whether to use signature presence test first to filter out those signatures that are not needed in the reconstruction of the spectrum.
<code>save.files</code>	If <code>TRUE</code> save several files for each input sample in a directory named after the sample.

Value

A list with the elements:

- `proposed.assignment`: Proposed signature assignment for `spectra` with the highest MAP found. If `use.sparse.assign = TRUE`, this will be the most sparse set of signatures that can plausibly explain `spectra`.
- `proposed.reconstruction`: Proposed reconstruction of `spectra` based on MAP. If `use.sparse.assign = TRUE`, this will be the reconstruction based on sparse assignment.
- `reconstruction.distances`: Various distances and similarities between `spectra` and `proposed.reconstruction`.
- `all.tested`: All tested possible ways to reconstruct each sample in `spectra`.
- `alt.solutions`: A tibble showing all the alternative solutions that are statistically as good as the `proposed.assignment` that can plausibly reconstruct `spectra`.
- `time.for.assignment`: Value from `system.time` for running `MAPAssignActivity1` for each sample in `spectra`.
- `error.messages`: Only appearing if there are errors running `MAPAssignActivity`.

The elements `proposed.assignment`, `proposed.reconstruction`, `reconstruction.distances`, `all.tested`, `time.for.assignment` will be `NULL` if the algorithm could not find the optimal reconstruction or there are errors coming out for **all** samples.

Examples

```
## Not run:
# This is a long running example unless parallel computing is supported on your machine
indices <- grep("Lung-AdenoCA", colnames(PCAWG7::spectra$PCAWG$SBS96))
spectra <- PCAWG7::spectra$PCAWG$SBS96[, indices[1:2], drop = FALSE]
SBS96.sigs <- cosmicSig::COSMIC_v3.2$signature$GRCh37$SBS96
sigs.prop <- ExposureProportions(mutation.type = "SBS96",
                                cancer.type = "Lung-AdenoCA")
sigs.to.use <- SBS96.sigs[, names(sigs.prop), drop = FALSE]
MAP.out <- MAPAssignActivity(spectra = spectra,
                            sigs = sigs.to.use,
                            sigs.presence.prop = sigs.prop,
                            output.dir = file.path(tempdir(), "Lung-AdenoCA"),
                            max.level = ncol(sigs.to.use) - 1,
                            p.thresh = 0.05 / ncol(sigs.to.use),
                            num.parallel.samples = 2,
                            mc.cores.per.sample = 30,
                            seed = 2561)

## End(Not run)
```

PresenceAttributeSigActivity

Find minimal set of signatures that can explain multiple spectra by first using signature presence test

Description

Find minimal set of signatures that can explain multiple spectra by first using signature presence test

Usage

```
PresenceAttributeSigActivity(
  spectra,
  sigs,
  output.dir,
  p.thresh = DefaultPThresh(sigs),
  m.opts = DefaultManyOpts(spectra = spectra),
  num.parallel.samples = 1,
  mc.cores.per.sample = 1,
  seed = 123,
  drop.low.mut.samples = FALSE,
  save.files = TRUE
)
```

Arguments

<code>spectra</code>	The spectra (multiple spectra) to be reconstructed.
<code>sigs</code>	A numerical matrix, possibly an ICAMS catalog.
<code>output.dir</code>	Directory path to save the output file.
<code>p.thresh</code>	If the p value for a better reconstruction with a set of signatures (as opposed to without that set of signatures) is > than this argument, then we can use exposures without this set.
<code>m.opts</code>	See DefaultManyOpts .
<code>num.parallel.samples</code>	The (maximum) number of samples to run in parallel. On Microsoft Windows machines it is silently changed to 1. Each sample in turn can require multiple cores, as governed by <code>mc.cores.per.sample</code> .
<code>mc.cores.per.sample</code>	The maximum number of cores to use for each sample. On Microsoft Windows machines it is silently changed to 1.
<code>seed</code>	Random seed; set this to get reproducible results. (The numerical optimization is in two phases; the first, global phase might rarely find different optima depending on the random seed.)
<code>drop.low.mut.samples</code>	Whether to exclude low mutation samples from the analysis. If TRUE, samples with SBS total mutations less than 100, DBS or ID total mutations less than 25 will be dropped.
<code>save.files</code>	If TRUE save several files for each input sample in a directory named after the sample.

Value

A list with the elements:

- `proposed.assignment`: The proposed set of signatures that can plausibly explain spectra.
- `proposed.reconstruction`: The reconstruction based on `proposed.assignment`.
- `reconstruction.distances`: Various distances and similarities between spectra and `proposed.reconstruction`.
- `time.for.assignment`: Value from `system.time` for running `PresenceAttributeSigActivity` for each sample in `spectra`.
- `error.messages`: Error messages running `PresenceAttributeSigActivity`.

Examples

```
## Not run:
# This is a long running example unless parallel computing is supported on your machine
indices <- grep("Lung-AdenoCA", colnames(PCAWG7::spectra$PCAWG$SBS96))
spectra <- PCAWG7::spectra$PCAWG$SBS96[, indices[1:2], drop = FALSE]
SBS96.sigs <- cosmic_v3.2::COSMIC_v3.2$signature$GRCh37$SBS96
sigs.prop <- ExposureProportions(
  mutation.type = "SBS96",
  cancer.type = "Lung-AdenoCA"
)
sigs.to.use <- SBS96.sigs[, names(sigs.prop), drop = FALSE]
retval <- PresenceAttributeSigActivity(
```

```
spectra = spectra,
sigs = sigs.to.use,
output.dir = file.path(tempdir(), "Lung-AdenoCA"),
num.parallel.samples = 2,
mc.cores.per.sample = 30,
seed = 2561
)

## End(Not run)
```

ReconstructSpectrum

Given signatures (sigs) and exposures (exp), return a spectrum or spectra

Description

Given signatures (sigs) and exposures (exp), return a spectrum or spectra

Usage

```
ReconstructSpectrum(sigs, exp, use.sig.names = FALSE)
```

Arguments

<code>sigs</code>	Signature as a matrix or data frame, with each row one mutation type (e.g. CCT > CAT or CC > TT) and each column a signature.
<code>exp</code>	The exposures for one or more samples as a matrix or data.frame, with each row a signature and each column a sample.
<code>use.sig.names</code>	If TRUE check that <code>rownames(exp)</code> is a subset of <code>colnames(sigs)</code> , and use only the columns in <code>sigs</code> that are present in <code>exp</code> .

Details

Does not care or check if `colSums(sigs) == 1`. Error checking is minimal since this function is called often.

Value

The matrix product `sig %*% exp` after some error checking.

Examples

[illegible]

SignaturePresenceTest

Test whether a given signature is plausibly present in a catalog.

Description

Test whether a given signature is plausibly present in a catalog.

Usage

```
SignaturePresenceTest (
  spectra,
  sigs,
  target.sig.index,
  m.opts = DefaultManyOpts(likelihood.dist = "multinom"),
  seed = NULL,
  mc.cores = 2
)
```

Arguments

spectra	The catalog (matrix) to analyze. This could be an ICAMS catalog or a numerical matrix.
sigs	A catalog of signatures from which to choose. This could be an ICAMS catalog or a numerical matrix.
target.sig.index	The index of the signature the presence of which we want to test. It can also be the signature id (e.g. "SBS22").
m.opts	See DefaultManyOpts .
seed	Random seed; set this to get reproducible results. (The numerical optimization is in two phases; the first, global phase might rarely find different optima depending on the random seed.)
mc.cores	Number of cores to use. Always silently changed to 1 on Microsoft Windows.

Value

A list of test results for each sample in `spectra`. Each sublist contains the following elements:

- `loglh.with`: The maximum log likelihood of the reconstructed spectrum using all the signatures.
- `loglh.without`: The maximum log likelihood of the reconstructed spectrum without the target signature.
- `statistic`: Likelihood ratio test statistic.
- `chisq.p`: P-value of the likelihood ratio test. The null hypothesis is we can plausibly reconstruct the spectrum without the target signature.
- `exp.with`: The exposure using all the signatures which generates the maximum log likelihood `loglh.with`.
- `exp.without`: The exposure not using the target signature which generates the maximum log likelihood `loglh.without`.

Examples

```

indices <- grep("Lung-AdenoCA", colnames(PCAWG7::spectra$PCAWG$SBS96))
spectra <- PCAWG7::spectra$PCAWG$SBS96[, indices[1:2], drop = FALSE]
sigs <- cosmicSig::COSMIC_v3.2$signature$GRCh37$SBS96
sigs.prop <- ExposureProportions(mutation.type = "SBS96",
                                cancer.type = "Lung-AdenoCA")
sigs.to.use <- sigs[, names(sigs.prop), drop = FALSE]
# Test whether SBS17a is plausibly present in the spectra
sig.presence.test.out <- SignaturePresenceTest(spectra = spectra,
                                              sigs = sigs.to.use,
                                              target.sig.index = "SBS17a",
                                              seed = 2581,
                                              mc.cores = 2)

```

SparseAssignActivity

Find known signatures that can most sparsely reconstruct each spectrum in a catalog.

Description

Find known signatures that can most sparsely reconstruct each spectrum in a catalog.

Usage

```

SparseAssignActivity(
  spectra,
  sigs,
  output.dir,
  max.level = 5,
  p.thresh = 0.05,
  m.opts = DefaultManyOpts(),
  num.parallel.samples = 5,
  mc.cores.per.sample = min(20, 2^max.level),
  progress.monitor = NULL,
  seed = NULL,
  max.subsets = 1000,
  drop.low.mut.samples = TRUE,
  use.sig.presence.test = FALSE
)

```

Arguments

<code>spectra</code>	The spectra (multiple spectra) to be reconstructed.
<code>sigs</code>	A numerical matrix, possibly an ICAMS catalog.
<code>output.dir</code>	Directory path to save the output file.
<code>max.level</code>	The maximum number of signatures to try removing.
<code>p.thresh</code>	If the p value for a better reconstruction with a set of signatures (as opposed to without that set of signatures) is > than this argument, then we can use exposures without this set.

<code>m.opts</code>	See DefaultManyOpts .
<code>num.parallel.samples</code>	The (maximum) number of samples to run in parallel. On Microsoft Windows machines it is silently changed to 1. Each sample in turn can require multiple cores, as governed by <code>mc.cores.per.sample</code> .
<code>mc.cores.per.sample</code>	The maximum number of cores to use for each sample. On Microsoft Windows machines it is silently changed to 1.
<code>progress.monitor</code>	Function called at the start of each new level (number of signatures to try excluding). Must take named arguments <code>value</code> and <code>detail</code> , and no others. Designed for a AsyncProgress progress bar function.
<code>seed</code>	Random seed; set this to get reproducible results. (The numerical optimization is in two phases; the first, global phase might rarely find different optima depending on the random seed.)
<code>max.subsets</code>	This argument provides a way to heuristically limit the amount of time spent by this function. Larger values of this argument will tend to allow longer running times. The algorithm successively tries to remove all subsets of 1 signature, 2 signatures, 3 signatures, etc., down to <code>max.level</code> . (Not every subset is tested at each level; if a subset was already found to be necessary the algorithm does not test supersets of that subset.) If at any level the algorithm needs to test more than <code>max.subsets</code> this function will not proceed.
<code>drop.low.mut.samples</code>	Whether to exclude low mutation samples from the analysis. If <code>TRUE</code> , samples with SBS total mutations less than 100, DBS or ID total mutations less than 25 will be dropped.
<code>use.sig.presence.test</code>	Whether to use signature presence test first to filter out those signatures that are not needed in the reconstruction of the spectrum.

Value

A list with the elements:

- `proposed.assignment`: The most sparse set of signatures that can plausibly explain spectra.
- `proposed.reconstruction`: The reconstruction based on sparse assignment.
- `reconstruction.distances`: Various distances and similarities between spectra and `proposed.reconstruction`.
- `all.tested`: All tested possible ways to reconstruct each sample in spectra.
- `alt.solutions`: A tibble showing all the alternative solutions that are statistically as good as the `proposed.assignment` that can plausibly reconstruct spectra.
- `time.for.assignment`: Value from `system.time` for running `SparseAssignActivity` for each sample in spectra.
- `error.messages`: Only appearing if there are errors running `SparseAssignActivity`.

The elements `proposed.assignment`, `proposed.reconstruction`, `reconstruction.distances`, `all.tested`, `time.for.assignment` will be `NULL` if the algorithm could not find the optimal reconstruction or there are errors coming out for **all** samples.

Examples

[illegible]

Index

AsyncProgress, [6](#), [12](#)

CancerTypes, [4](#)

cossim, [2](#)

DefaultManyOpts, [3](#), [5](#), [8](#), [10](#), [12](#)

ExposureProportions, [4](#), [5](#)

ICAMS, [3](#), [5](#), [8](#), [10](#), [11](#)

MAPAssignActivity, [5](#)

NegBinomial, [3](#)

nloptr, [3](#)

PresenceAttributeSigActivity, [7](#)

ReconstructSpectrum, [9](#)

SignaturePresenceTest, [10](#)

SparseAssignActivity, [11](#)